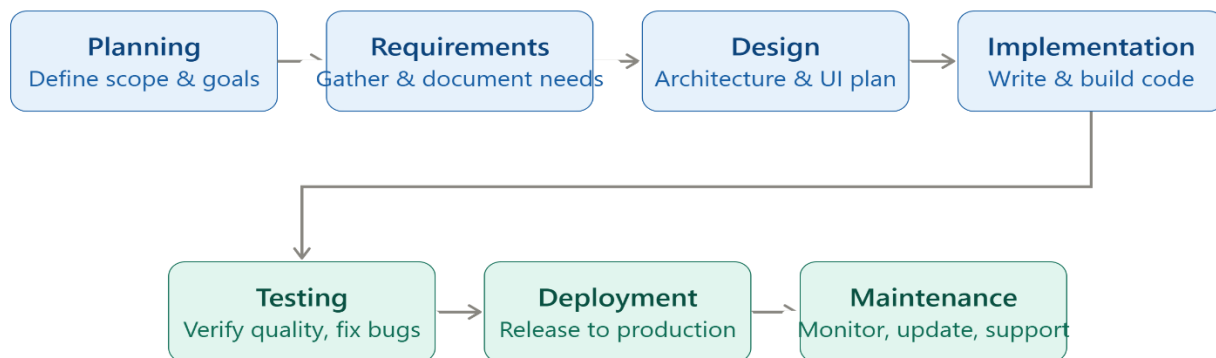


Q1. SDLC Lifecycle

(DIAGRAM DRAWING)



SDLC Lifecycle

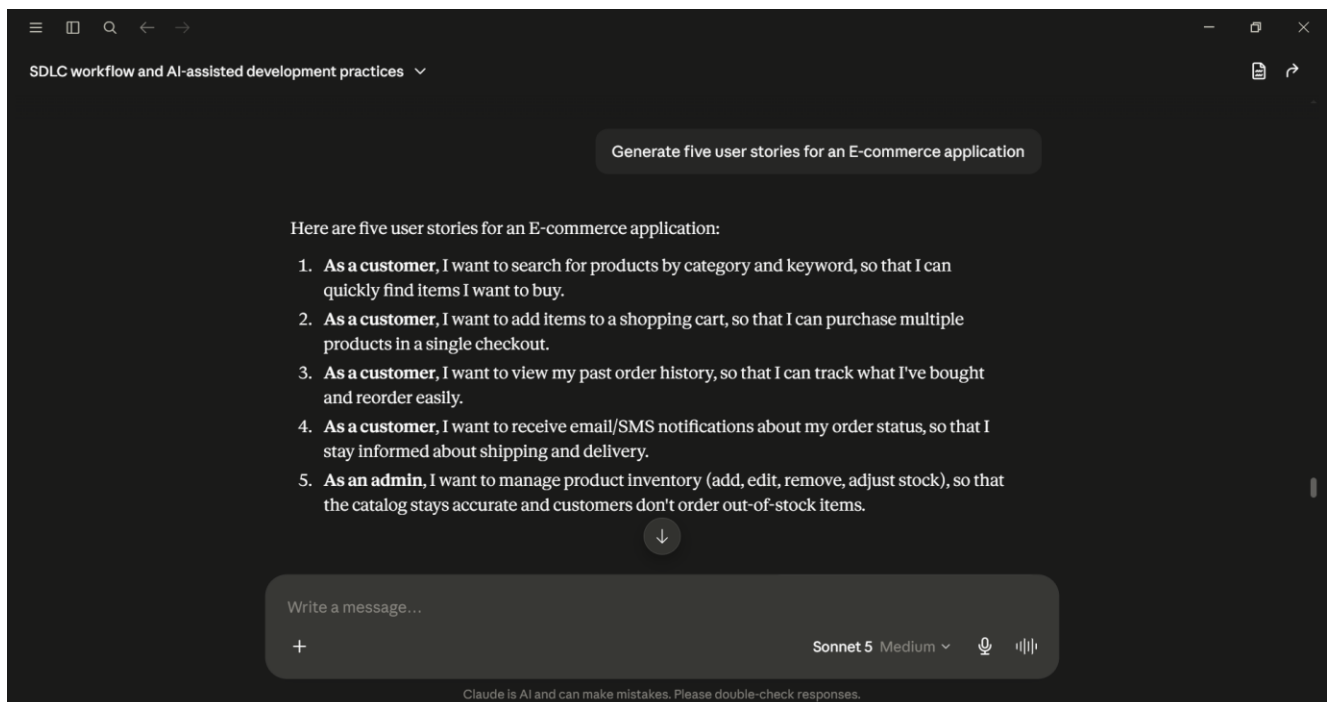
The diagram above shows the seven stages most teams recognize, though some models group them into 5-6. Here's how the workflow moves:

- **Requirement Analysis** – Requirement Analysis phase is one of the most important phases of SDLC. During this phase developer team talks to stakeholders and users to gather, document, and prioritize functional and non-functional requirements. In this phase important documents are BRD (Business Requirement Document), SRS (Software Requirement Specification), User cases, RTM (Requirement Traceability Matrix).
- **Design** – This phase translates requirements into system architecture, database schema, UI/UX wireframes, and technical specifications. Here the activities are HLD (High-Level Design), LLD (Low-Level Design), Database Design, API Design, etc.
- **Implementation** – Developers write the actual code according to the design and coding standards, including Coding Review, Unit Testing, Version Control (Git), Integration Test, CI/CD Integration.
- **Testing** – QA verifies the software against requirements: unit, integration, system, and user acceptance testing to catch defects.
- **Deployment** – The verified build is released to staging and then production, often via CI/CD pipelines where end users can access it. Activities are production release, Database Migration, configuration updates, Smoke Testing, Monitoring, Rollback Planning.
- **Maintenance** – The team monitors the live system, fixes bugs, applies patches, and rolls out enhancements — feedback from this stage loops back into the next round of Planning, which is why modern SDLC is iterative (Agile/DevOps) rather than a strict one-way waterfall.

Q2. Practical AI Exercise using Claude AI

Here are five user stories for an E-commerce application:

1. **As a customer**, I want to search for products by category and keyword, so that I can quickly find items I want to buy.
2. **As a customer**, I want to add items to a shopping cart, so that I can purchase multiple products in a single checkout.
3. **As a customer**, I want to view my past order history, so that I can track what I've bought and reorder easily.
4. **As a customer**, I want to receive email/SMS notifications about my order status, so that I stay informed about shipping and delivery.
5. **As an admin**, I want to manage product inventory (add, edit, remove, adjust stock), so that the catalog stays accurate and customers don't order out-of-stock items.



Q2. Practical AI Exercise using Claude AI

Story converted into acceptance criteria:

As a customer, I want to add items to a shopping cart, so that I can purchase multiple products in one Checkout.

1. **Given** I'm on a product page, when I click "Add to Cart," then the item is added and the cart icon updates its item count immediately.
2. **Given** an item is already in my cart, when I add the same item again, then its quantity increases instead of creating a duplicate line entry.
3. **Given** I have items in my cart, when I open the cart view, then I see each product's name, image, unit price, quantity, and line subtotal, plus an order total.
4. **Given** a product is out of stock, when I try to add it to my cart, then the system blocks the action and shows a clear "out of stock" message.
5. **Given** items are in my cart, when I remove an item or change its quantity, then the cart total and item count update instantly without a page reload.
6. **Given** I add items while not logged in, **when** I later log in, **then** my cart contents persist and merge with any existing saved cart

Q2. Practical AI Exercise using Claude AI

Sprint Backlog — E-commerce Application

#	User Story	Task	Estimate (SP)	Priority	Status
1	As a customer, I want to search products by category/keyword	Build search bar UI + backend query endpoint	5	High	To Do
1	As a customer, I want to search products by category/keyword	Add category & keyword filters	3	Medium	To Do
2	As a customer, I want to add items to a shopping cart	Cart data model + add/update/remove endpoints	5	High	To Do
2	As a customer, I want to add items to a shopping cart	Cart UI (icon, drawer, quantity controls)	5	High	To Do
2	As a customer, I want to add items to a shopping cart	Stock validation on add-to-cart	2	High	To Do
3	As a customer, I want to view my order history	Order history page + API	5	Medium	To Do
4	As a customer, I want order status notifications	Email/SMS integration for status changes	8	Medium	To Do
5	As an admin, I want to manage product inventory	Admin CRUD screens for products/stock	8	High	To Do

Sprint summary:

- Total effort: ~41 story points
- Sprint duration: 2 weeks (assuming a team velocity of ~20–25 SP/sprint, this backlog likely spans **two sprints**)
- Sprint 1 focus (recommended): Stories 2 and 1 — cart and search are the core purchase-path features and should be prioritized first
- Sprint 2 focus: Stories 5, 3, and 4 — inventory management, order history, and notifications

Q2. Practical AI Exercise using Claude AI

Review of the AI-generated output

What the AI did well

- Produced grammatically correct user stories in proper "As a / I want / so that" format without needing extra prompting.
- Wrote acceptance criteria in clean Given/When/Then structure, and covered several realistic edge cases (duplicate items, out-of-stock, quantity changes) on the first attempt.
- Organized the Sprint Backlog into a clear, usable table with story points, priority, and status — a reasonable starting structure a real team could work from.
- Kept consistent terminology and formatting across all four outputs, which made them easy to combine into one deliverable.
- list didn't account for guest users. I added a 6th criterion: *"Given I add items while not logged in, when I later log in, then my cart contents persist and merge with any existing saved cart."* This is a common real-world requirement the AI didn't surface unprompted.
- **Underestimated story points** : The AI initially estimated "Email/SMS integration for status changes" at a low value. I raised it to 8 SP, because third-party notification service integration (webhooks, retries, failure handling, template management) is routinely underestimated by AI, which has no visibility into a team's actual vendor setup or past incidents with similar integrations.
- **Priority reordering** : The AI ranked "Order history" as high priority alongside checkout-related tasks. I moved it to Medium, since it doesn't block a purchase, and re-prioritized cart validation and search higher instead, since those directly affect revenue and conversion.
- **Vague task naming** : Some AI-generated task titles (e.g., "handle cart logic") were too generic to assign to a developer without follow-up questions. I rewrote them into specific, actionable tasks .
- **Sprint grouping** : The AI listed all tasks flatly without splitting them across sprints. I added a two-sprint breakdown based on typical team velocity (20–25 SP/sprint), since 41 total story points is too much for one sprint.

Q2. Practical AI Exercise using Claude AI

Corrections made manually

. **Missing edge case in acceptance criteria** : The AI's first cart-related acceptance criteria
Why human review is still necessary

- **No knowledge of team context** : The AI doesn't know your team's actual velocity, current tech debt, or skill gaps, so its estimates and sprint groupings are generic guesses, not calibrated numbers.
- **Business priorities require judgment** : Deciding what's "High" vs "Medium" priority is a business decision tied to revenue impact, compliance deadlines, and stakeholder commitments — context the AI simply doesn't have access to.
- **Domain-specific edge cases get missed** :Real e-commerce systems have nuances (tax rules, regional shipping, loyalty programs, fraud checks) that only a domain expert or product owner would know to ask for; the AI can only generate what's common/generic.
- **Acceptance criteria are a draft, not a contract** : AI-written acceptance criteria are a strong starting point, but treating them as final without review risks false confidence in test coverage — someone still has to validate they match the real business rules.
- **Accountability** : Ultimately, a human (product owner, scrum master, or lead developer) has to own the final backlog, since they're accountable for what the team commits to and delivers — the AI has no stake in the outcome.